2D Videohra Salvation Path: Herní mechaniky

2D Video game Salvation Path: Game mechanics

Maturitní práce
Vedoucí práce: Bc. Adam Tyroň
2025/26
Daniel Baier, 4.D

Abstrakt

Tato práce se zabývá vývojem videohry Surreal Bowling: Salvation Path a patří do skupiny tří maturitních prací ve třídě 4.D, které jsou na tuto videohru zaměřeny. Náš nezávislý tým se jmenuje Still Thinking Studio. Všechny tři maturitní práce řeší z většiny programovací a logickou část toho, jak videohra funguje. Moje práce se zaměřujě na programování mechanik pro pohyb. Součástí práce je také implementace fyzikálního systému nebo systému střelby. Dále se v této práci zaměřuji na programování logiky nepřítele a nastavení adekvátních animací pro postavy.

Klíčová slova

videohra, 2D plošinovka, Unity, C#, fyzika, animace

Abstract

This thesis examines the development of the video game Surreal Bowling: Salvation Path and is one of three graduation theses in class 4.D that focus on this particular video game. Our independent team is referred to as Still Thinking Studio. The three graduation projects primarily address the programming and logical components of the video game's functionality. My project focuses on programming movement mechanics. It also includes the implementation of a physics system and a shooting system. Additionally, this project focuses on programming enemy AI and setting up appropriate animations for the characters.

Key words

video game, 2D platformer, Unity, C#, physics, animations

Poděkování

Rád bych poděkoval panu Bc. Adamu Tyroňovi za víceletou aktivitu na Discord serveru našeho vývojářského týmu a ujmutí se nás jako mentor maturitní práce. Dále chci samozřejmě poděkovat svým spolupracovníkům a zároveň spolužákům, s kterými videohry vyvíjím - Simeonu Bednarzovi za vyvíjení kódu, Jonáši Kymlovi za zajištění audiovizuální stránky naší hry a Jakubu Wojtasovi za téměř veškeré grafické zpracování.

Prohlášení

Prohlašuji, že jsem svou maturitní práci vypracoval/a samostatně. K práci jsem použil/a literaturu a prameny, uvedené v seznamu. Souhlasím s tím, aby moje ročníková práce byla využívána na Střední průmyslové škole Karviná, příspěvkové organizaci.

V Karviné dne: ……. Podpis: …….

Obsah

Obsah 4

1 Terminologie 7
1.1 Definice pojmů . . . 7
1.1.1 Collider . . . 7
1.1.2 Rigidbody . . . 7
1.1.3 Player controller . . . 7
1.1.4 Input action . . . 7
1.1.5 Sprite . . . 7
1.1.6 Animace . . . 7
1.1.7 Animator controller . . . 8
1.1.8 Tilemap . . . 8
1.1.9 Tileset . . . 8
1.1.10 Game feel . . . 8
1.1.11 Frame . . . 8
1.1.12 Layer . . . 8
1.1.13 Prefab . . . 9
1.1.14 ScriptableObject . . . 9
1.1.15 Callback . . . 9

2 Použitý framework aplikace 10
2.1 Použitý framework aplikace . . . 10
2.1.1 Operační systém . . . 10
2.1.2 Herní engine . . . 10
2.1.3 Programovací jazyk . . . 11
2.1.4 IDE / Code editor . . . 11
2.1.5 Kolaborace . . . 11

3 Celková koncepce řešení 12
3.1 Koncepce řešení . . . 12
3.1.1 Základní pohyb . . . 12
3.1.2 Rozšiřující funkce . . . 12
3.1.3 Systém střelby . . . 14
3.1.4 Animace . . . 15
3.1.5 Fyzika a gravitace . . . 15

4 Popis aplikace **16**
4.1 Aplikace 16
4.1.1 PlayerControllerData 16
4.1.2 PlayerController 19
4.1.3 Skok 26
4.1.4 Dash 30
4.1.5 Wall jump a wall slide 33
4.1.6 Střelba 37
4.1.7 Bowlingová koule 41
4.1.8 Další skripty nepřítele 47

Seznam obrázků **51**

Úvod

Cílem této práce je navrhnout a implementovat herní mechaniky pro postavy videohry žánru 2D plošinovky. Práce se zaměřuje na systémy ovládání hráče a návrh logiky nepřítele. Mechaniky hráče nezahrnují pouze pohyb do stran, ale také systém skoku a střelby. Zvláštní důraz je kladen na implementaci skoku, jehož správné nastavení zásadně ovlivňuje herní pocit a odlišuje kvalitní 2D plošinovky od méně povedených titulů.

Součástí práce je také implementace fyzikálního systému. Herní engine Unity sice nabízí vestavěné nástroje pro práci s fyzikou, ty však nemusí vždy poskytovat požadovanou míru kontroly nad chováním postavy. Z tohoto důvodu jsou tyto nástroje využity především jako základ, zatímco pokročilejší herní mechaniky jsou řešeny vlastním kódem.

S programováním herních mechanik úzce souvisí také implementace animací. Po vytvoření grafických podkladů je nutné jednotlivé animace správně integrovat do enginu a propojit je s řídicí logikou tak, aby odpovídaly aktuálnímu stavu postavy.

Významnou část projektu tvoří rovněž systém střelby založen na automatickém míření a výstřelu při inputu hráče. Tento systém přináší řadu technických výzev, například správné určení směru projektilu, úpravu jeho rychlosti nebo zajištění jeho odstranění po určité době z důvodu optimalizace výkonu hry.

1 Terminologie

1.1 Definice pojmů

1.1.1 Collider

Collider je komponenta v Unity, která umožňuje přidat kolize na objekty uvnitř hry. Díky colliderům se dá poté například určovat ničení herních objektů nebo zabíjení nepřátel. Také lze colliderům nastavit funkci *Trigger*, která se používá například při aktivování jiných scén ve hře nebo při aktivaci interakcí. *Trigger collider* nezastaví hráče při kolizi, ale detekuje průchod a následně spustí požadovanou akci nebo kód.

1.1.2 Rigidbody

Rigidbody je komponenta v Unity, která umožňuje herním objektům používat fyzikální engine hry. Díky manipulaci herních objektů upravováním parametrů v Rigidbody je možné dělat realistické kolize, upravovat gravitaci nebo celkově nastavit chování herních objektů.

1.1.3 Player controller

Označení *Player controller* se nejčastěji používá pro hlavní skript, který se stará o ovládání hráče. Do tohoto skriptu se nejčastěji přidávají funkce pro chůzi, skákání nebo jiné pokročilejší mechaniky, které jsou specifické pro danou hru.

1.1.4 Input action

Input action je speciální komponent v Unity, který řeší input z klávesnice, myši nebo třeba controlleru pro konzole. Dříve se programoval input za pomoci `Input.GetKey(KeyCode._)`, což nebylo vyloženě špatné, ovšem input action je funkční zjednodušení tohoto systému.

1.1.5 Sprite

Sprite je grafický asset vytvořený ve specializovaném programu. Jedná se o komponenty, které člověk uvidí ve výsledném produktu.

1.1.6 Animace

Animace je složení několika spritů za sebe, které po aktivaci navozují pocit pohybu. Lze u nich upravovat rychlost a délku.

1.1.7 Animator controller

Animator controller neboli zkráceně *Animator* se využívá ke spojování animací ve hře a vytváření hierarchie mezi nimi. Ve spolupráci s kódem lze nastavovat spouštění specifických animací při specifických příležitostech.

1.1.8 Tilemap

Tilemap je metoda tvorby herního prostředí pomocí pravidelné mřížky složené z grafických dlaždic (tiles). Každá dlaždice představuje samostatný grafický prvek, který je umístěn do určité buňky mřížky. Tento systém se používá především ve 2D hrách, kde umožňuje efektivní a přehlednou tvorbu rozsáhlých herních map. Výhodou tilemap systému je možnost opakovaného použití grafických prvků, snížení paměťové náročnosti a snadná úprava úrovní.

1.1.9 Tileset

Tileset je soubor grafických dlaždic určených k sestavení herního prostředí. Obvykle je tvořen jedním obrázkem (sprite sheet), který obsahuje více dlaždic stejné velikosti uspořádaných vedle sebe. Použití tilesetu umožňuje efektivní práci s grafikou, protože jednotlivé prvky lze opakovaně využívat při tvorbě mapy.

1.1.10 Game feel

Game feel neboli herní pocit je termín, který popisuje, jak hra působí na hráče z hlediska ovládání a reakcí. Zahrnuje faktory jako je responsivita ovládání, plynulost animací, zpětná vazba od hry a celkový dojem z interakce s herním světem. Kvalitní herní pocit je to, co odlišuje průměrné hry od těch výborných.

1.1.11 Frame

Frame neboli snímek je jednotka času v herním cyklu. Hry typicky běží při 30 až 60 snímcích za sekundu, což znamená, že každou sekundu se herní logika a grafika aktualizují 30 až 60 krát. Pochopení framu je důležité pro implementaci herních mechanik, protože mnoho systémů (jako coyote time nebo jump buffer) pracuje právě s časovým intervalem měřeným ve snímcích.

1.1.12 Layer

Layer neboli vrstva je systém v Unity pro organizaci a filtrování objektů ve scéně. Umožňuje například určit, které objekty mají detekovat kolize s jinými objekty. V kontextu platformovek se často hovoří o *ground layer*, což je vrstva obsahující všechny platformy a zem, na které může hráč stát.

1.1.13 Prefab

Prefab (prefabricated object) je předem vytvořený herní objekt uložený jako asset v projektu. Umožňuje vytvořit instanci (kopii) objektu přímo v kódu pomocí metody
`Instantiate`. Prefaby se často používají pro projektil, nepřátele nebo jiné opakovaně vytvářené objekty ve hře.

1.1.14 ScriptableObject

ScriptableObject je speciální datový typ v Unity určený pro ukládání konfiguračních dat a parametrů. Na rozdíl od `MonoBehaviour` není přidělen k žádnému hernímu objektu ve scéně a může existovat jako samostatný asset. Používá se typicky pro nastavení herních parametrů, které je možné měnit v editoru bez nutnosti upravovat kód.

1.1.15 Callback

Callback je mechanismus v programování, kdy se jako parametr předá jiná funkce, která se má zavolat po dokončení určité operace. V kontextu Unity input systému se callbacky používají k reagování na události, jako je stisknutí nebo uvolnění klávesy. Když hráč stiskne tlačítko, systém automaticky zavolá předem definovanou metodu (callback).

2 Použitý framework aplikace

2.1 Použitý framework aplikace

2.1.1 Operační systém

Pro zajištění co největší efektivity je důležité si vybrat správný operační systém. Samozřejmě se nabízí možnost použít Windows, který berou vývojáři assetů a doplňků nejvíce v potaz, jelikož je to stále nejpoužívanější systém pro běžné uživatele. Ovšem u tohoto systému je obrovský problém s optimalizací a efektivitou. Windows má otevřenou spoustu aplikací na pozadí, které značně brzdí celkový výkon počítače. Co se týče práce s více okny, Windows na tom není vyloženě špatně, ovšem pro můj způsob práce je nejlepší práce s více plochami spíše než s více okny na jedné ploše, a v tom je Windows neskutečně pomalý. Zkusil jsem proto najít ideální systém pro můj způsob práce, přešel jsem proto na stranu Linuxu a FOSS. Vyzkoušel jsem několik různých distribucí Linuxu, než jsem našel tu pravou pro mě. Začal jsem na Ubuntu, prošel jsem přes CentOS, Pop!_OS a Fedoru, až jsem se nakonec dostal k legendární distribuci Arch. Tento systém je pro mou práci ideální v kombinaci s Wayland compositorem Hyprland. Jeho rychlost je mnohonásobně vyšší než u Windows a práce s plochami je velmi svižná a příjemná.

2.1.2 Herní engine

https://www.g2.com/glossary/game-engines-definition

Herní engine dává vývojářům framework pro vývoj her bez nutnosti vytváření všech systémů, jako například fyziku, grafiku nebo AI od úplného začátku. Herní engine odstraňuje potřebu programovat a integrovat systémy pro podporu hry, tímto šetří čas a zdroje vývojářů, aby se mohli soustředit na tvorbu samotné hry. Na trhu je několik různých možností enginů na výběr. Mezi nejznámější patří třeba Unreal Engine 5, Unity Engine nebo Godot. Tento projekt používá Unity Engine z několika důvodů. Unity je stále jeden z nejpopulárnějších enginů současnosti, díky tomu existuje spousta materiálů, které může člověk při vývoji použít. Unity se zároveň dá dobře použít jak pro 3D, tak pro 2D projekty, na rozdíl od Unreal Engine 5, který se specializuje především na 3D. Další důležité kritérium při výběru herního enginu je jazyk, který se používá pro programování. Unreal používá C++, Godot má možnost použít jejich vlastní GDScript, který se nejvíce podobá jazyku Python, ale nabízí také možnost použít C#, C nebo C++ díky GDExtension. Unity používá defaultně C#, což je zároveň jazyk, který se učím už 4 roky díky škole. Bohužel v době vybírání enginu ještě Godot nebyl ani zdaleka tak pokročilý, jako je v dnešní době. Kdyby byl Godot už tehdy stabilní, tak by nejspíše byl využit i pro tento projekt. Ovšem díky době, komunitě, jazyku a možnostem byl pro tento projekt zvolen engine Unity.

2.1.3 Programovací jazyk

https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/

V herních enginech se programuje hlavně pomocí programovacích jazyků. Avšak třeba Unreal nabízí možnost programovat čistě pomocí bloků, kterými se nastavuje chování a spojují se mezi sebou. Vzhledem k tomu, že tento projekt jede na Unity, tak používá jazyk C#. C# je moderní a intuitivní programovací jazyk, který vývojářům umožňuje vytvářet mnoho typů bezpečných a stabilních aplikací. Jeho kořeny sahají do rodiny jazyků C.

2.1.4 IDE / Code editor

IDE (Integrated Development Environment) je komplexní vývojové prostředí určené pro programování. Spojuje v sobě editor kódu, kompilátor nebo interpret, debugger a další nástroje potřebné pro vývoj aplikací. Umožňuje programátorovi psát, spouštět, testovat a ladit programy na jednom místě. Typickými příklady jsou například Visual Studio nebo IntelliJ IDEA.

Code editor je jednodušší nástroj sloužící především k psaní a úpravě zdrojového kódu. Obvykle nabízí zvýrazňování syntaxe, automatické doplňování kódu a možnost instalace rozšíření. Na rozdíl od IDE většinou neobsahuje všechny pokročilé nástroje pro ladění a správu projektu v základní výbavě. Mezi známé editory patří například Visual Studio Code nebo Sublime Text.

Zpočátku jsem používal Visual Studio 2022. To bylo při začátcích na systému Windows. Při přechodu na Linux jsem však zároveň přešel na Visual Studio Code, do kterého jsem pouze stáhl pár rozšíření pro programování v Unity. Práce ve VS Code pro mě byla znatelně jednodušší a přehlednější, proto bych už nepřešel zpět k Visual Studiu.

2.1.5 Kolaborace

https://git-scm.com/

Vzhledem k tomu, že pracujeme ve více lidech na jednom projektu, bylo potřeba vybrat způsob kolaborace. Unity nabízí svou vlastní možnost, neboli Unity Version Control. Ovšem poté, co jsme to jednou zkusili použít, jsme od toho dali ruce pryč a zapřísáhli se, že už to nikdy nepoužijeme. Proto používáme Git a GitHub.

Git je bezplatný, snadno použitelný systém pro správu zdrojového kódu, který je navržen pro rychlé a efektivní zpracování menších i velmi rozsáhlých projektů. Git software disponuje nízkou technickou náročností a velmi vysokou rychlostí, čímž díky možnostem lokálního větvení a alternativních verzí zajišťuje svižný průběh jakékoli programátorské práce.

GitHub je webové rozhraní, které umožňuje více lidem současně provádět změny na jednom projektu pomocí softwaru Git. GitHub nabízí spolupráci v reálném čase a je proto nezbytným rozšířením systému Git.

3 Celková koncepce řešení

3.1 Koncepce řešení

Nejprve bylo nutné stanovit základní koncept videohry. Bez něj není možné začít programovat mechaniky. Projekt vznikl v rámci game jamu, tedy časově omezené soutěže zaměřené na vývoj hry na zadané téma. S týmem jsme se rozhodli udělat hru o kuželce, která utíká před masivní bowlingovou koulí.

Na základě tohoto konceptu bylo možné navrhnout základní funkčnost hlavních herních prvků, neboli kuželky jako hráčem ovládané postavy a bowlingové koule jako hrozby. U bowlingové koule jsme zvažovali dvě varianty řešení. První možností bylo implementovat naváděcí mechanismus, který by sledoval aktuální polohu hráče a dynamicky upravoval trajektorii pohybu. Druhou možností bylo ponechat kouli v předem definovaném směru pohybu, avšak přizpůsobit její velikost a návrh úrovně tak, aby i bez aktivního navádění představovala dostatečné nebezpečí.

S ohledem na časové omezení soutěže jsme zvolili druhé řešení, což nám umožnilo věnovat více prostoru implementaci klíčových mechanik hráče. Po skončení soutěže jsme toto rozhodnutí zachovali, jelikož masivní nenaváděná koule působí herně i vizuálně
výrazněji a přirozeněji než přesně naváděný objekt.

Následovala implementace mechanik hráče, které lze rozdělit do tří hlavních oblastí: základní pohyb, rozšiřující prvky zlepšující ovladatelnost a systém střelby.

3.1.1 Základní pohyb

Prvním krokem bylo vytvoření základního pohybu, na který bylo možné později navázat pokročilejší funkce. Tento základ zahrnuje horizontální pohyb (vlevo a vpravo) a základní logiku skoku, tedy vertikální pohyb. V této fázi bylo nutné implementovat zpracování vstupu (inputu) hráče s následnou reakcí herního objektu ve scéně. Základní část tedy spočívá v jednoduchém pohybu podmíněném inputem uživatele.

3.1.2 Rozšiřující funkce

Druhým krokem bylo rozšíření základních mechanik o prvky zvyšující plynulost a komfort ovládání. Klíčovou oblastí této fáze byl systém skoku. V základní implementaci může hráč vyskočit pouze v okamžiku, kdy se dotýká země, což je řešeno pomocí vrstev (layers) a kolizemi (collisions). V praxi však hráč často dává input ke skoku s mírným zpožděním, například v momentě, kdy již technicky opustil platformu. Bez dodatečných úprav by v takové situaci ke skoku nedošlo, což by vedlo k nepřesnému a frustrujícímu ovládání. Z tohoto důvodu jsem implementoval mechanismy označované jako *Coyote time* a *Jump buffer*.

Coyote time

Coyote time dovolí hráči skočit do vzduchu i o pár snímků (framů) dále, než je aktivní ground layer. Za předpokladu, že chce hráč skočit z platformy a už je technicky za ní, tak by měl spadnout, ovšem *Coyote time* dává hráči malé okno neperfektnosti, kdy se skok stále zaregistruje, i když už by hráč měl správně spadnout.

Jump buffer

Jump buffer funguje podobně jako *Coyote time*. Rozdíl je ten, že *Jump buffer* se zaměřuje na skok při dopadu na zem. Když hra zaznamená input od hráče v momentě, kdy hráč teprve dopadá z výšky na zem, ale ještě na ní není, avšak je pár framů nad ní, tak se tento input opět zaregistruje jako skok a herní objekt se posune na vertikální ose.

Wall jump a wall slide

Další implementovanou mechanikou bylo skákání a klouzání po stěnách, tedy *Wall jump* a *Wall slide*. Tyto dvě funkce spolu úzce souvisejí. *Wall jump* umožňuje hráči opětovně vyskočit při kontaktu se stěnou ze strany, čímž se otevírá možnost postupného pohybu mezi dvěma vertikálními plochami a dosažení vyšších částí úrovně.

Wall slide doplňuje předchozí mechaniku tím, že upravuje vertikální rychlost při kontaktu se stěnou. Bez této úpravy by hráč podléhal standardní gravitaci a rychlosti a neměl by dostatečný časový prostor pro provedení dalšího skoku. *Wall slide* proto zpomaluje pád podél stěny a umožňuje kontrolovanější vertikální pohyb. Díky tomu se dá teoreticky nastavit, aby hráč nemohl vůbec spadnout a byl *přilepený* na stěně, což se k některým hrám může hodit, ovšem my jsme se rozhodli pro to, aby hráč stále padal i na stěně.

Variable jump height

https://selfdeterminationtheory.org/SDT/documents/2010_PrzybylskiRigbyRyan_ROGP.pdf

https://www.void1gaming.com/post/flow-theory-in-game-design

O této funkci tolik lidí neví, jelikož není na první pohled tak výrazná jako třeba *Wall jump*. To však nic nemění na tom, jak signifikantní rozdíl v uživatelské přívětivosti dělá. Zjednodušeně jde o funkci, která zajišťuje, že když hráč při skoku drží input pro skok delší dobu, tak jeho postava vyskočí výše, než kdyby input jen rychle stiskl. Ve hrách se to často používá právě proto, že jsou někdy v úrovních skoky, kde hráč potřebuje skočit spíše nízko, jinak by například zemřel při kolizi se stropem. Zároveň to podporuje lidskou přirozenost držet input déle při atmosféricky napjatých situacích. Když hráč potřebuje přeskočit delší propast, tak bude delší dobu držet input pro skok.

Dash

Posledním krokem je mechanika rychlého výpadu, neboli *Dash*. Jde o funkci, kdy se herní objekt na krátký časový okamžik rapidně zrychlí a posune do aktuálního směru pohybu při inputu uživatele. Tato mechanika je sice celkem jednoduchá svým zněním, ovšem při hraní samotné hry velmi zvyšuje plynulost a celkově zpříjemňuje pocit z pohybu.

3.1.3 Systém střelby

Systém střelby se dá rozdělit na 3 hlavní části. Jsou jimi míření (aim), vystřelení a chování samotného projektilu.

Míření (aim)

Je více možností, jak vyřešit aim. Zprvu jsme se rozhodli uspořádat aim k pohybu kurzoru myši. Tohle nám dovolovalo přidávat další objekty pro interakce do scény. Ovšem žádné takové objekty jsme na game jamu nestihli implementovat. Následně jsme začali přemýšlet nad přidáváním dalších schopností (ability), které by udělali naši hru zajímavější. To však vyžaduje více inputu na klávesnici. Kdybychom přidali nějaký ability, hráč by měl velmi aktivní pouze jednu ruku (pro pohyb a ability) a ruka na myši by se téměř nepoužívala, jelikož by byla potřeba čistě na časy, kdy přijede blízko bowlingová koule. Proto jsem tento systém předělal tak, že je aim nastaven automaticky na objekt koule ve scéně, a hráči stačí input na klávesnici pro výstřel. Tohle nám dovoluje přidat více různých schopností pro hráče, jelikož obě ruce můžou být teď aktivní na klávesnici, a střelba se bude považovat za další abilitu.

Výstřel

Samotný výstřel se dá řešit více způsoby. Buď může být nastaveno automatické spuštění po uběhnutí časového intervalu, nebo se může nastavit automatický výstřel pouze při objevení se nepřátelského objektu na obrazovce s případným offsetem. My jsme však zvolili nejčastější možnost, která je dle nás i nejlepší. Hráč má výstřel kompletně ve svých rukou. Nemusí se spoléhat na žádný automatický systém, který by ho mohl v průběhu hry *zradit*, protože by nevystřelil v čas, kdy hráč chce.

Chování projektilu

Logika pro projektil se kompletně odděluje od skriptu pro střílení. Je to nejlepší způsob, jak řešit optimalizaci střílení, jelikož se skript vystřeleného projektilu spustí pouze při samotném výstřelu a není celou dobu aktivní ve skriptu pro střelbu. Zároveň se díky tomu jednodušeji vytváří chování projektilu. V naší hře jsme se rozhodli, aby střelba byla jen poslední možností záchrany pro hráče. Na rozdíl od většiny her, kde se střílí, u nás není možné zabít nebo zničit nepřátelský objekt. Naše projektily pouze zpomalují kouli na nižší rychlost. Ovšem poté, co se koule opět zrychlí, tak se jí

nastaví vyšší rychlost, než byla před zpomalením. Zároveň má hráč k dispozici pouze 2 náboje, které se přebíjejí až po vystřelení obou a po krátkém časovém intervalu. Tímto podporujeme naši myšlenku, aby výstřel byla jen poslední možnost záchrany a hráč se musel spíše soustředit na zlepšování se v pohybu a skocích.

3.1.4 Animace

Moje práce nebyla pouze o programování mechanik hráče a nepřítele. Musel jsem také nastavovat animace těmto objektům. Použil jsem assety vytvořené Jakubem Wojtasem (členem a hlavním grafikem našeho týmu) a už je pouze nastavoval v Unity. Zde bylo důležité správně si rozdělit animace a nastavit jim podmínky spuštění v Animatoru a poté je správně spouštět z kódu. Nesmí se stát, že hráč bude mít animaci běhu, přestože stojí na místě.

3.1.5 Fyzika a gravitace

Unity nabízí vestavěné komponenty pro upravování gravitace a fyziky objektů. Pro základ a začátky jsou sice dostačující, ovšem pro realistický a uživatelsky přívětivý systém je potřeba vzít tyto základní komponenty, vynulovat jejich hodnoty a udělat vlastní skripty. To nabízí možnost dynamicky upravovat gravitaci podle aktuálních podmínek ve hře. Pokud chce hráč praktikovat *Wall jump*, tak se gravitace trochu sníží, aby hráč nespadl příliš rychle. Zároveň lze znatelně více upravovat fyziku skoku. Ve hrách je ideální, aby trochu déle trvalo, než se hráč dostane do maximální výšky skoku, poté je dobré, aby hráč chvíli zůstal *viset* ve vzduchu a až potom rychle spadl. Tyto věci by nešly udělat ani zdaleka tak jednoduše přes vestavěné komponenty v Unity.

4 Popis aplikace

4.1 Aplikace

Tato sekce se podrobně zabývá implementací jednotlivých herních mechanik na úrovni kódu. Budou popsány klíčové třídy, jejich metody a algoritmy, které tvoří základ herního systému.

4.1.1 PlayerControllerData

Třída `PlayerControllerData` představuje klíčový prvek celého systému herních mechanik, neboť definuje veškeré parametry pohybu a chování hráče na jednom centrálním místě. V této sekci bude podrobně rozebrána struktura této třídy, její účel a způsob použití v rámci celého systému.

Účel a využití ScriptableObject

V Unity je možné vytvořit více druhů skriptů. Nejčastěji používaným je `MonoBehaviour`, který je určen především pro skripty přidělované na herní objekty v scéně. Pro ukládání dat a konfigurační parametry však není `MonoBehaviour` ideální volbou, protože vyžaduje přítomnost herního objektu a není možné jej snadno sdílet mezi více scénami.

Z tohoto důvodu existuje typ `ScriptableObject`, který je navržen speciálně pro ukládání dat a konfiguračních hodnot. `ScriptableObject` je samostatný datový objekt, který není přidělen k žádnému hernímu objektu a může být vytvořen jako samostatný asset v projektu. Hlavní výhodou tohoto přístupu je možnost mít více různých sad parametrů a snadno je mezi nimi přepínat. Každý `ScriptableObject` asset může obsahovat odlišné hodnoty, což umožňuje například vytvořit různé profily obtížnosti nebo testovací scénáře.

Pro vytvoření `ScriptableObject` ve editoru se používá atribut `[CreateAssetMenu]`, který přidá položku do menu pro vytvoření nového assetu. Tím získá developer možnost tweakovat hodnoty bez nutnosti otevírat samotný skript a měnit v něm konstanty. Při změně hodnot v `ScriptableObject` assetu se tyto změny okamžitě projeví ve hře, což výrazně zrychluje proces ladění herního pocitu.

```
[CreateAssetMenu(menuName = "Player Movement Data")]
public class PlayerControllerData : ScriptableObject
{
    [Header("Run")]
    public float runMaxSpeed = 8f;
    public float runAcceleration = 20f;
    [HideInInspector] public float runAccelAmount;

    [Header("Jump")]
    public float jumpForce = 15f;
    public float coyoteTime = 0.1f;
    public float jumpBufferTime = 0.1f;

    [Header("Gravity")]
    public float gravity = 9.8f;
    public float fallMultiplier = 2.5f;

    [Header("Dash")]
    public float dashSpeed = 20f;
    public float dashTime = 0.15f;
    public float dashCooldown = 0.3f;

    [Header("Wall Slide / Jump")]
    public float wallSlideSpeed = 2f;
    public float wallJumpDuration = 0.4f;
```

```
    public float wallJumpPowerY = 16f;
}
```

Kód 4.1: Listing 1: Definice třídy PlayerControllerData

Struktura dat a organizace

Třída `PlayerControllerData` dědí z `ScriptableObject` a je organizována do několika sekcí, z nichž každá sdružuje parametry související s konkrétní herní mechanikou. Pro přehledné oddělení jednotlivých sekcí v editoru se používá atribut `[Header]`, který vytváří vizuální oddělovače mezi skupinami proměnných.

První sekce obsahuje parametry pro běh (run), konkrétně maximální rychlost, zrychlení a poměr zrychlení ve vzduchu. Druhá sekce se zaměřuje na skok a obsahuje sílu skoku, časové parametry pro coyote time a jump buffer. Třetí sekce definuje parametry gravitace, včetně základní gravitace a multiplikátoru pro pád. Čtvrtá sekce obsahuje nastavení pro přerušení skoku (jump cut) a pátá sekce parametry pro dash. Poslední sekce se věnuje wall jump, včetně sil pro jednotlivé typy skoků.

Tato organizace umožňuje rychle nalézt a upravit specifický parametr bez nutnosti procházet neuspořádaný seznam všech proměnných. Zároveň usnadňuje pochopení systému jako celku, protože jednotlivé sekce jasně korespondují s implementovanými herními mechanikami.

Důležité parametry a jejich vliv na herní pocit

Každý parametr v `PlayerControllerData` má specifický vliv na chování hráče a celkový herní pocit. Pochopení jejich vzájemného působení je klíčové pro správné nastavení mechanik.

Mezi nejdůležitější parametry běhu patří `runMaxSpeed`, který definuje maximální rychlost horizontálního pohybu. Parametr `runAcceleration` určuje, jak rychle hráč dosáhne této maximální rychlosti. Vysoká hodnota zrychlení vede k okamžitému startu, zatímco nízká hodnota vytváří efekt rozjezdu. Důležitým parametrem je také `accelInAir`, který určuje procentuální poměr pozemního zrychlení použitého ve vzduchu. Nízká hodnota tohoto parametru ztěžuje změnu směru během skoku, což může být žádoucí pro obtížnější sekce.

Pro skok jsou klíčové parametry `jumpForce` určující sílu počátečního výskoku, `coyoteTime` definující dobu grace period po opuštění platformy, a `jumpBufferTime` pro registraci skoku před dopadem. Parametr `jumpHangTimeThreshold` určuje prahovou rychlost, při které se aktivuje efekt vznášení na vrcholu skoku.

Gravitace je řízena parametrem `gravity` pro základní gravitační sílu a `fallMultiplier` pro zrychlení pádu. Vyšší hodnota fall multiplieru vede k rychlejšímu pádu a kratšímu času ve vzduchu, což zlepšuje kontrolu nad přesným načasováním dopadu.

Dash je konfigurován pomocí `dashSpeed` pro rychlost výpadu, `dashTime` pro dobu trvání a `dashCooldown` pro prodlevu mezi dashi.

Výpočet akcelerace a její automatizace

Pro zajištění konzistentního chování pohybu bez ohledu na maximální rychlost se v metodě `OnValidate()` automaticky počítá hodnota `runAccelAmount` z parametrů `runAcceleration` a `runMaxSpeed`. Tento výpočet je důležitý pro zajištění toho, aby zrychlení bylo proporcionální k maximální rychlosti.

Vztah mezi akcelerací a maximální rychlostí je následující: při nízké maximální rychlosti a vysoké akceleraci by mohl hráč dosáhnout maximální rychlosti příliš rychle, což by vedlo k neintuitivnímu chování. Automatickým výpočtem se zajistí, že pohyb zůstává konzistentní a předvídatelný pro jakoukoli kombinaci hodnot.

Metoda `OnValidate()` se volá v editoru při změně hodnot v inspektoru, což znamená, že výpočet probíhá automaticky při každé úpravě parametrů. Nemusíme tak manuálně počítat správnou hodnotu akcelerace a může se soustředit pouze na nastavení požadovaného chování.

Použití v PlayerController a propojení systémů

V třídě `PlayerController` je instance `PlayerControllerData` přidělena jako serializované pole pomocí atributu `[SerializeField]`. Toto pole se v editoru zobrazuje jako pole pro přetahování (drag and drop), kam lze umístit libovolný `ScriptableObject` asset vytvořený z třídy `PlayerControllerData`.

Při spuštění hry se v metodě `Awake()` načtou všechny potřebné reference a inicializují se výchozí hodnoty. Parametry z `PlayerControllerData` se pak používají v celém systému pro řízení chování hráče. Toto propojení umožňuje snadné ladění hodnot bez nutnosti otevírat kód a měnit konstanty přímo ve skriptech.

Výhodou tohoto přístupu je možnost vytvořit více variant `PlayerControllerData` assetů s různými parametry. Například lze vytvořit jeden asset pro testovací úroveň s nižší obtížností a jiný pro finální úroveň s vyššími nároky na ovládání. Přepínání mezi těmito variantami je pak otázkou jediného přetahovacího pole v inspektoru.

4.1.2 PlayerController

Hlavním řídicím skriptem celého systému pohybu hráče je třída `PlayerController`. Tento skript funguje jako centrální uzel, který koordinuje všechny ostatní systémy a zajišťuje jejich vzájemnou spolupráci. V této sekci budou popsány jednotlivé aspekty implementace tohoto skriptu, včetně zpracování vstupu, aplikace pohybu a propojení s dalšími systémy.

Hlavní skript a jeho role v systému

Třída `PlayerController` představuje hlavní skript propojující všechny systémy pohybu hráče. Dědí z `MonoBehaviour` a přidává se na herní objekt hráče jako komponenta. Tento skript funguje jako centrální uzel, který koordinuje vstupy, fyziku, gravitaci a propojení s dalšími systémy jako dash, střelba a wall jump.

V metodě `Awake()` probíhá inicializace skriptu. Získávají se reference na komponentu `Rigidbody2D`, která je esenciální pro fyzikální pohyb, a na ostatní skripty jako `PlayerDash`, `PlayerStripeShot` a `PlayerWall`. Důležitým krokem je nastavení `gravityScale` na hodnotu 0 na komponentě `Rigidbody2D`. To je klíčové pro fungování vlastního gravitačního systému, protože Unity vestavěná gravitace je kompletně deaktivována a veškerá gravitace je řešena vlastním kódem v metodě `ApplyCustomGravity()`.

```
private void Awake()
{
    rb = GetComponent<Rigidbody2D>();
    rb.gravityScale = 0;
    isFacingRight = true;
    pd = GetComponent<PlayerDash>();
    pss = GetComponent<PlayerStripeShot>();
    pw = GetComponent<PlayerWall>();
}
```

Kód 4.2: Listing 2: Awake metoda

Input systém a zpracování uživatelského vstupu

Vstupy od uživatele se zpracovávají pomocí Unity Input Systemu, který poskytuje modernější a flexibilnější přístup ke zpracování vstupů oproti staršímu Input Manageru. Pomocí atributu `[SerializeField] private InputActionReference` se přiřazují jednotlivé akce v editoru. Každá akce repreguje konkrétní typ vstupu, jako je pohyb, skok nebo střelba.

K registraci inputu dochází v metodě `OnEnable()`, kde se připojují callbacky na jednotlivé akce. Tyto callbacky jsou metody, které se automaticky volají při změně stavu vstupu. Příkladem je `OnMoveInput()` pro pohyb, `OnJumpInput()` pro skok a podobně. Při uvolnění skriptu nebo deaktivaci objektu se v `OnDisable()` tyto callbacky odpojují, což zabraňuje chybám při volání metod na neexistující objekty.

Tento přístup k input systému je výhodný, protože umožňuje snadnou rekonfiguraci vstupů bez nutnosti měnit kód. V editoru lze přiřadit libovolnou klávesu nebo tlačítko k akci a systém automaticky zpracuje změnu.

```
[Header("input")]
public Vector2 moveInput { get; private set; }
[SerializeField] private InputActionReference moveAction;
[SerializeField] private InputActionReference jumpAction;
[SerializeField] private InputActionReference dashAction;
[SerializeField] private InputActionReference shootAction;

private void OnEnable()
{
    moveAction.action.Enable();
    jumpAction.action.Enable();
    shootAction.action.Enable();
    dashAction.action.Enable();

    jumpAction.action.performed += OnJumpInput;
    jumpAction.action.canceled += OnJumpRelease;
    dashAction.action.performed += OnDashInput;
    shootAction.action.performed += OnShootInput;
}
```

Kód 4.3: Listing 3: Deklarace vstupních akcí

Zpracování vstupu a filtrování hodnot

V metodě `Update()` se čte vstup z akce pohybu a následně filtruje pomocí prahu 0.1. Tento práh slouží k ignorování nechtěného inputu z joysticku, který může generovat velmi malé hodnoty i při neutrální pozici. Bez tohoto filtru by se hráč mohl nechtěně pohybovat i když joystick není aktivně ovládán. Tato logika platí také pro input z klávesnice.

Hodnota vstupu se ukládá jako `Vector2`, který obsahuje složky x a y. Podle hodnoty na ose x se následně volá metoda `CheckDirectionToFace()`, která kontroluje otočení postavy. Pokud je vstup kladný a postava se aktuálně dívala doleva, postava se otočí doprava. Podobná logika funguje také v opačném scénáři. Toto zajišťuje, že postava vždy směřuje ve směru pohybu.

Metoda `Update()` je vhodná pro zpracování vstupu, protože běží jednou za snímek a není vázána na fyzikální timestep. To umožňuje responsivní zpracování inputu bez ohledu na fyzikální nastavení hry.

Horizontální pohyb a akcelerace

Metoda `Run()` aplikuje horizontální pohyb pomocí `Lerp` interpolace, která zajišťuje plynulé zrychlení a zpomalování namísto okamžité změny rychlosti. Vzorec `Mathf.Lerp(rb.linearVelocity.x, targetSpeed, Time.fixedDeltaTime * accelRate)` postupně přesouvá aktuální rychlost k cílové rychlosti.

Zrychlení se liší podle toho, zda je hráč na zemi nebo ve vzduchu. Parametr `accelInAir` určuje, kolik procent pozemního zrychlení se použije ve vzduchu. Typicky se používá nižší hodnota (například 50-70 procent), což zajišťuje, že hráč nemůže ve vzduchu měnit směr tak rychle jako na zemi. Toto omezení přidává hloubku do herní mechaniky a odměňuje hráče za správné načasování pohybu.

Během provádění wall jump je volání `Run()` dočasně deaktivováno, aby hráč nemohl během letu po stěně reagovat na směrové tlačítko. Toto zajišťuje, že wall jump má konzistentní trajektorii bez ohledu na input hráče během letu.

```
private void Run()
{
    if (pw != null && pw.IsWallJumping)
        return;
    if (rb == null || data == null) return;

    float targetSpeed = moveInput.x * data.runMaxSpeed;

    float accelRate = lastOnGroundTime > 0 ?
        data.runAccelAmount :
        data.runAccelAmount * data.accelInAir;

    if (isJumping && Mathf.Abs(rb.linearVelocity.y) < data.
       jumpHangTimeThreshold)
    {
        accelRate *= data.jumpHangAccelerationMult;
        targetSpeed *= data.jumpHangMaxSpeedMult;
    }

    float newVelX = Mathf.Lerp(rb.linearVelocity.x, targetSpeed,
        Time.fixedDeltaTime * accelRate);
    rb.linearVelocity = new Vector2(newVelX, rb.linearVelocity.y);
}
```

Kód 4.4: Listing 4: Metoda Run pro horizontální pohyb

Vlastní gravitace a její dynamické řízení

Jelikož je `gravityScale` na `Rigidbody` nastaven na 0, gravitace se kompletně řeší vlastním kódem v metodě `ApplyCustomGravity()`. Toto řešení poskytuje plnou kontrolu nad chováním hráče ve vzduchu a umožňuje dynamické úpravy gravitace podle aktuálního stavu.

Metoda rozlišuje čtyři různé stavy, z nichž každý má specifickou gravitační sílu. Prvním stavem je normální pád dolů, při kterém se použije `fallMultiplier` pro rychlejší pád. To zajistí, že hráč padá rychleji, než stoupá, což je typické pro kvalitní platformovky. Druhým stavem je přerušení skoku (jump cut), kdy hráč pustí tlačítko skoku během vzestupu. V tomto případě se použije ještě vyšší multiplikátor gravitace pro okamžité ukončení vzestupu. Třetím stavem je dosažení vrcholu skoku, kdy vertikální rychlost klesne pod `jumpHangTimeThreshold`. V tomto momentě se gravitace dočasně sníží, což vytváří efekt krátkého vznášení. Posledním stavem je normální stav bez jakékoli modifikace, kdy se použije základní gravitace.

```
private void ApplyCustomGravity()
{
    if (rb == null || data == null) return;

    float gravity = data.gravity;
    Vector2 vel = rb.linearVelocity;

    if (vel.y < 0)
    {
        vel.y += Physics2D.gravity.y * gravity *
            data.fallMultiplier * Time.fixedDeltaTime;
    }
    else if (jumpCut)
    {
        vel.y += Physics2D.gravity.y * gravity *
            data.jumpCutMultiplier * Time.fixedDeltaTime;
    }
    else if (isJumping && Mathf.Abs(vel.y) < data.jumpHangTimeThreshold
        )
    {
        vel.y += Physics2D.gravity.y * gravity *
            data.jumpHangGravityMult * Time.fixedDeltaTime;
    }
    else
    {
        vel.y += Physics2D.gravity.y * gravity *
            Time.fixedDeltaTime;
```

```
    }

    rb.linearVelocity = vel;
}
```

Kód 4.5: Listing 5: Metoda ApplyCustomGravity

Výpočet nové rychlosti a použití Lerp

Při aplikaci pohybu se používá interpolace pomocí `Lerp` pro zajištění plynulého zrychlení a zpomalování. Tento přístup je výhodnější než přímé nastavení rychlosti, protože vytváří přirozenější herní pocit. Hráč nedosáhne maximální rychlosti okamžitě, ale postupně se k ní přibližuje.

Vzorec `Mathf.Lerp(rb.linearVelocity.x, targetSpeed, Time.fixedDeltaTime * accelRate)` pracuje následovně: první parametr je aktuální rychlost na ose x, druhý parametr je cílová rychlost (buď maximální rychlost ve směru pohybu nebo 0 při zastavení) a třetí parametr určuje rychlost interpolace. Parametr `Time.fixedDeltaTime` zajišťuje, že interpolace je nezávislá na snímkové frekvenci, zatímco `accelRate` určuje, jak rychle se má hráč accelerovat.

Hodnota `targetSpeed` se vypočítá vynásobením vstupu (1 nebo -1) s maximální rychlostí. To zajišťuje, že hráč běží maximální rychlostí, pokud drží směrové tlačítko, a zastaví se, pokud žádné tlačítko nedrží.

Zemní kontrola a detekce dotyku se zemí

Kontrola, zda se hráč dotýká země, je fundamentální pro správné fungování skoku a dalších mechanik. V `PlayerController` se tato kontrola provádí metodou `IsGrounded()`, která využívá `Physics2D.OverlapBox`.

Metoda `Physics2D.OverlapBox` vytváří virtuální box na specifikované pozici a kontroluje, zda se v tomto prostoru nachází nějaký collider. Parametry zahrnují
`groundCheckPoint` udávající střed boxu a `groundCheckSize` udávající rozměry boxu. Box je typicky malý a umístěný na spodní část nohou hráče.

Výsledek detekce se ukládá do proměnné `grounded`, která se následně používá pro rozhodování, zda je možné provést skok, jakou akceleraci použít a další logické operace. Detekce probíhá v každém snímku, což zajišťuje aktuální stav bez zpoždění.

Propojení s Animatorem a předávání parametrů

Animace jsou důležitou součástí herního pocitu a přispívají k celkovému dojmu ze hry. V metodě `Update()` se předávají hodnoty do Animatoru, který následně přepíná mezi jednotlivými stavy animací podle aktuálního stavu hráče.

Prvním předávaným parametrem je rychlost na ose x (`speed`), která se používá pro animaci běhu. Hodnota se typicky pohybuje od 0 do 1 a Animator ji používá k přechodu mezi stavy idle

a run, případně k nastavení rychlosti animace.

Druhým parametrem je `isGrounded`, což je boolean hodnota indikující, zda se hráč dotýká země. Tento parametr se používá pro přepínání mezi animací stání a animací vzduchu (skok, pád). Třetím parametrem je vertikální rychlost (`verticalVelocity`), která slouží k detekci pádu. Pokud je vertikální rychlost záporná a hráč není na zemi, spustí se animace pádu.

Správné propojení mezi logikou a animacemi je klíčové pro plynulý herní zážitek a eliminaci vizuálních nesrovnalostí mezi chováním postavy a její animací.

Otočení postavy a vizuální změna směru

Metoda `CheckDirectionToFace()` kontroluje, zda se postava má otočit, a provádí vizuální změnu směru. Na rozdíl od fyzikální změny směru, která by vyžadovala změnu vektoru rychlosti, tato metoda pouze mění vizuální prezentaci postavy.

Otočení se provádí překlopením `localScale.x` na zápornou hodnotu. Pokud je postava otočená doprava a vstup je doleva (záporná hodnota), `localScale.x` se nastaví na -1. Podobně při otočení doleva a vstupu doprava se `localScale.x` nastaví na 1.

Tento přístup je výhodný, protože nevyžaduje změnu fyzikálního směru, který je řízen rychlostí na `Rigidbody2D`. Postava vizuálně směřuje správným směrem, zatímco fyzika může být řešena nezávisle. To je užitečné například při střelbě, kde projektil vždy směřuje k cíli bez ohledu na vizuální orientaci postavy.

Propojení s dash systémem a deaktivace pohybu

V metodě `FixedUpdate()` třídy `PlayerController` se kontroluje, zda je aktivní dash. Pokud ano, přeskočí se volání `Run()` a `ApplyCustomGravity()`. Toto zajišťuje, že během dashe hráč nereaguje na gravitaci ani na vstup pohybu.

Důvod pro deaktivaci gravitace během dashe je ten, že dash by měl být konzistentní bez ohledu na vertikální polohu. Pokud by gravitace byla stále aktivní, hráč by během dashe klesal, což by narušilo dojem rychlého výpadu. Deaktivace vstupu pohybu zajišťuje, že dash má předem definovanou trajektorii a hráč ji nemůže během dashe měnit.

Propojení mezi `PlayerController` a `PlayerDash` je implementováno pomocí volání metody `IsDashing()` v `PlayerDash`, která vrací aktuální stav dashe. Toto rozhodnutí se pak používá v `FixedUpdate()` pro přeskočení standardních herních mechanik.

4.1.3 Skok

Systém skoku představuje jednu z nejdůležitějších součástí každé 2D plošinovky a jeho správná implementace zásadně ovlivňuje celkový herní pocit. V této sekci budou podrobně rozebrány všechny implementované mechaniky související se skokem, včetně pokročilých technik jako je coyote time, jump buffer a proměnná výška skoku.

Coyote time a jeho implementace

Jak už bylo zmíněno v koncepci, coyote time dovolí hráči skočit i pár snímků poté, co opustil platformu. V `PlayerController` se to řeší pomocí proměnné `lastOnGroundTime`, která se při dopadu na zem nastaví na hodnotu `coyoteTime` z `PlayerControllerData`. Následně se v každém snímku v metodě `Update()` tato proměnná snižuje o `Time.deltaTime`.

V momentě, kdy hráč stiskne skok, se kontroluje, zda je `lastOnGroundTime` větší než 0. Pokud ano, skok se provede i když hráč technicky už není na zemi. Toto řešení poskytuje hráči malé okno neperfektnosti, kdy se skok stále zaregistruje, i když už by hráč měl správně spadnout.

Hodnota `coyoteTime` se typicky nastavuje na 0.1 až 0.2 sekundy, což odpovídá přibližně 6-12 snímkům při 60 snímcích za sekundu. Tato doba je dostatečně krátká, aby neovlivnila běžnou hru, ale dostatečně dlouhá, aby zachytila nepatrně opožděný input hráče.

```
// In Update() the timer decreases
private void Update()
{
    lastOnGroundTime -= Time.deltaTime;
    lastPressedJumpTime -= Time.deltaTime;
    IsGrounded();

    // Set coyote time when grounded
    if (grounded)
        lastOnGroundTime = data.coyoteTime;

    // Check jump with coyote time
    if (lastPressedJumpTime > 0 && lastOnGroundTime > 0 && !pw.
       IsWallJumping)
        Jump();
}
```

Kód 4.6: Listing 6: Implementace coyote time

Jump buffer a registrace skoku před dopadem

Jump buffer funguje obdobně jako coyote time, ale na opačnou stranu časové osy. zatímco coyote time řeší situaci po opuštění platformy, jump buffer řeší situaci před dopadem na zem. Když hra zaznamená input od hráče v momentě, kdy hráč teprve dopadá z výšky na zem, ale ještě na ní není, avšak je pár snímků nad ní, tento input se zaregistruje jako skok.

Implementace využívá proměnnou `lastPressedJumpTime`, která se nastaví při stisknutí skoku na hodnotu `jumpBufferTime` z `PlayerControllerData`. V každém snímku se tato hodnota snižuje o `Time.deltaTime`.

V `Update()` se pak kontroluje, zda je `lastPressedJumpTime > 0` a současně

`lastOnGroundTime > 0`. Pokud obě podmínky platí, provede se skok. Toto řešení eliminuje frustraci z toho, že hráč stiskne skok těsně před dopadem a skok se neprovede, protože hráč ještě není na zemi.

Provádění skoku a nastavení fyzikálních parametrů

Metoda `Jump()` se volá, když jsou splněny všechny podmínky pro skok (dostatečný coyote time a jump buffer). Nejprve se nastaví `isJumping` na true, což indikuje, že hráč je aktuálně ve vzduchu a nelze skočit znovu. Současně se nastaví `jumpCut` na false, což resetuje stav přerušení skoku.

Následně se vynuluje vertikální rychlost na `Rigidbody2D` a poté se nastaví na hodnotu `jumpForce` z `PlayerControllerData`. Toto zajistí konzistentní počáteční rychlost skoku bez ohledu na předchozí pohyb. Důležité je také vynulování časových proměnných `lastOnGroundTime` a `lastPressedJumpTime`, aby se skok neopakoval v dalších snímcích.

Hodnota `jumpForce` se typicky pohybuje v rozmezí 10-20 v závislosti na nastavení gravitace a požadované výšce skoku. Správné nastavení této hodnoty je klíčové pro celkový herní pocit.

```
private void Jump()
{
    if (rb == null || data == null) return;

    isJumping = true;
    jumpCut = false;
    lastPressedJumpTime = 0;
    lastOnGroundTime = 0;

    rb.linearVelocity = new Vector2(rb.linearVelocity.x, 0);
    rb.linearVelocity = new Vector2(rb.linearVelocity.x, data.jumpForce
        );
}
```

Kód 4.7: Listing 7: Metoda Jump pro provedení skoku

Jump cut a předčasné ukončení skoku

Při puštění tlačítka skoku během vzestupu se aktivuje jump cut, který předčasně ukončí skok a umožní hráči rychleji spadnout. Toto je důležitá mechanika, která umožňuje hráči kontrolovat výšku skoku podle délky stisknutí tlačítka.

Implementace využívá metodu `OnJumpRelease()`, která se registruje jako callback při uvolnění vstupu skoku. V této metodě se kontroluje, zda je hráčova vertikální rychlost kladná (tedy stoupá). Pokud ano, vynásobí se rychlost hodnotou `jumpCutVelocityMultiplier` z `PlayerControllerData`. Typicky se používá hodnota 0.5, což znamená, že při puštění tlačítka se rychlost sníží na polovinu.

V metodě `ApplyCustomGravity()` se pak při aktivním `jumpCut` aplikuje ještě vyšší gravitace než při normálním pádu. Toto zajistí okamžité ukončení vzestupu a rychlý přechod do pádu, což přispívá k responsivitě ovládání.

```
private void OnJumpRelease(InputAction.CallbackContext ctx)
{
    if (rb != null && rb.linearVelocity.y > 0)
    {
        jumpCut = true;

        rb.linearVelocity = new Vector2(
            rb.linearVelocity.x,
            rb.linearVelocity.y * data.jumpCutVelocityMultiplier
        );
    }
}
```

Kód 4.8: Listing 8: Metoda OnJumpRelease pro jump cut

Jump hang a efekt vznášení na vrcholu skoku

Po dosažení vrcholu skoku, kdy je vertikální rychlost nižší než `jumpHangTimeThreshold` z `PlayerControllerDa` se dočasně sníží gravitace. Tím vzniká krátký moment vznášení, který výrazně zlepšuje herní pocit a poskytuje hráči více času na načasování dalších akcí.

Implementace probíhá v metodě `ApplyCustomGravity()`, kde se kontroluje, zda je aktivní skok a zda je vertikální rychlost nižší než práh. Pokud ano, použije se snížená gravitace. Zároveň se v metodě `Run()` detekuje tento stav a mírně se zvýší akcelerace a maximální rychlost.

Toto zvýšení mobility během jump hang poskytuje hráči lepší kontrolu nad pohybem ve vzduchu v blízkosti vrcholu skoku. Hráč může lépe reagovat na herní situace a přesněji se dostat na cílovou platformu.

Vstup skoku a registrace callbacků

Skok se registruje jako input callback v metodě `OnJumpInput()`, která se připojí k Unity Input System akci skoku. V této metodě se nastavuje `lastPressedJumpTime` na hodnotu `jumpBufferTime`, což aktivuje jump buffer systém.

Proměnná `jumpPressedThisFrame` se nastaví na true, což indikuje, že v aktuálním snímku byl stisknut skok. Tato proměnná se používá pro okamžitou detekci stisknutí v rámci snímku. Na konci `Update()` se `jumpPressedThisFrame` vynuluje, aby se zabránilo opakovanému skoku v dalších snímcích.

Při uvolnění tlačítka skoku se volá `OnJumpRelease()`, která aktivuje `jumpCut`. Toto rozdělení na stisk a uvolnění umožňuje jemnou kontrolu nad výškou skoku a přispívá k celkové responsivitě ovládání.

4.1.4 Dash

Mechanika rychlého výpadu neboli dash představuje další důležitý prvek pohybového systému. Dash umožňuje hráči na krátký časový okamžik rapidní zrychlení a přesun do aktuálního směru pohybu. Tato mechanika při správném nastavení výrazně zvyšuje plynulost hry a celkově zpříjemňuje pocit z pohybu.

Základní princip a dědění z PlayerAbility

Mechanika rychlého výpadu je implementována ve třídě `PlayerDash`, která dědí ze třídy `PlayerAbility`. `PlayerAbility` je abstraktní třída, která poskytuje základní systém pro odemykání a zamykání jednotlivých schopností. Všechny ability skripty v projektu dědí z této abstraktní třídy.

Třída `PlayerAbility` obsahuje pouze proměnnou `unlocked` typu boolean a metody `Unlock()` a `Lock()`. Při volání `Unlock()` se `unlocked` nastaví na true a schopnost je k dispozici. Při volání `Lock()` se schopnost deaktivuje a hráč ji nemůže používat.

Tato architektura umožňuje snadné přidávání nových schopností bez nutnosti opakovat kód pro odemykání a zamykání. Každá konkrétní schopnost pouze implementuje svou vlastní logiku, zatímco systém odemykání je společný.

Metoda TryDash a kontrola podmínek

Metoda `TryDash()` je veřejná metoda volaná z `PlayerController` při inputu dashe. Nejprve kontroluje, zda je schopnost odemčená voláním `IsUnlocked()`. Pokud není, metoda se ukončí a dash se neprovede.

Následně se kontroluje cooldown a stav dashe. Pokud běží cooldown (`dashCooldownTimer > 0`) nebo pokud je dash aktuálně aktivní, metoda se ukončí. Teprve pokud všechny tyto podmínky jsou splněny, pokračuje se k samotnému provedení dashe.

Po úspěšné kontrole podmínek se určí směr dashe. Pokud hráč dává vstup pohybu (`moveInput` není nulový), použije se tento směr. V opačném případě se použije směr podle otočení postavy jako záloha, aby dash vždy fungoval i když hráč stojí na místě.

```
public void TryDash()
{
    if (!unlocked) return;

    if (isDashing || dashCooldownTimer > 0 || rb == null || data ==
       null) return;

    Vector2 moveInput = pc != null ? pc.moveInput : Vector2.zero;
    dashDir = new Vector2(moveInput.x, moveInput.y).normalized;
    if (dashDir == Vector2.zero)
        dashDir = pc.isFacingRight ? Vector2.right : Vector2.left;

    isDashing = true;
    dashTimer = data.dashTime;
    dashCooldownTimer = data.dashCooldown;
    rb.linearVelocity = dashDir * data.dashSpeed;
}
```

Kód 4.9: Listing 9: Metoda TryDash

Směr dashe a normalizace vektoru

Směr dashe se normalizuje, aby byl vždy jednotkový vektor. To zajišťuje, že dash má konzistentní rychlost bez ohledu na úhel vstupu. Pokud by normalizace nebyla provedena, diagonální dash by byl rychlejší než horizontální nebo vertikální.

Normalizace se provádí metodou `Normalize()` na `Vector2`. Pokud je vektor příliš krátký (méně než 0.01), považuje se za nulový a nepokračuje se v dashi.

V případě, že hráč nedává žádný vstup pohybu, použije se směr podle otočení postavy. To se zjistí pomocí proměnné `pc.isFacingRight`, která vrací true pokud se postava dívá doprava. Výsledný směr je pak `Vector2.right` nebo `Vector2.left`. Toto zajišťuje, že dash vždy funguje, i když hráč

stojí na místě a nedává žádný vstup.

DashMovement a aplikace rychlosti během dashe

Po dobu trvání dashe se ve `FixedUpdate()` přepisuje rychlost `Rigidbody` na konstantní hodnotu `dashDir * dashSpeed`. Toto zajišťuje, že hráč se pohybuje konstantní rychlostí bez ohledu na vstup nebo gravitaci. Vzorec je `rb.linearVelocity = new Vector2(dashDir.x * dashSpeed, dashDir.y * dashSpeed)`.

Po uplynutí `dashTime` se dash ukončí voláním `EndDash()`. Zároveň se nastaví cooldown timer na hodnotu `dashCooldown`. Po celou dobu trvání dashe se `Rigidbody` chová jako by měl nulovou hmotnost, protože rychlost je přepisována každý snímek. To znamená, že kolize během dashe fungují správně, ale hráč je řízen čistě logikou dashe.

```
private void FixedUpdate()
{
    if (isDashing)
    {
        DashMovement();
    }
}

private void DashMovement()
{
    if (rb == null || data == null) return;

    dashTimer -= Time.fixedDeltaTime;
    rb.linearVelocity = dashDir * data.dashSpeed;

    if (dashTimer <= 0)
    {
        isDashing = false;
    }
}

public bool IsDashing()
{
    return isDashing;
}
```

Kód 4.10: Listing 10: Metoda DashMovement

Cooldown systém a omezení frekvence dashe

Po dokončení dashe se nastaví `dashCooldownTimer` na hodnotu `dashCooldown` z `PlayerControllerData`. V `Update()` se tento timer snižuje o `Time.deltaTime`. Dash lze provést pouze když je timer menší nebo roven nule.

Tento cooldown systém zabraňuje spamování dashe a vyvažuje herní mechaniku. Bez cooldownu by hráč mohl dash používat téměř neustále, což by změnilo charakter hry. Správně nastavený cooldown nutí hráče strategicky využívat dash a rozhodovat se, kdy je nejvhodnější ho použít.

Hodnota `dashCooldown` se typicky nastavuje na 0.5-2 sekundy v závislosti na požadované obtížnosti a délce úrovní. Kratší cooldown umožňuje agresivnější hru, zatímco delší cooldown vyžaduje opatrnější přístup.

Propojení s PlayerController a registrace inputu

Ve `FixedUpdate()` třídy `PlayerController` se při aktivním dashi přeskočí volání `Run()` a `ApplyCustomGravity` jak bylo popsáno v předchozí sekci. Toto propojení zajišťuje, že všechny herní systémy spolupracují a nedochází ke konfliktům.

Dash se registruje jako input callback v metodě `OnDashInput()`, kde se volá `pd.TryDash()`. `pd` je reference na instanci `PlayerDash`, která se získá v `Awake()`. Metoda `IsDashing()` vrací aktuální stav dashe pro externí skripty, které potřebují vědět, zda hráč právě dashuje.

Toto propojení umožňuje čistou separaci kódu. `PlayerController` se stará o základní pohyb a gravitaci, zatímco `PlayerDash` se stará o specifickou logiku dashe. Spolupráce mezi těmito systémy je řízena přes veřejné metody a vlastnosti.

4.1.5 Wall jump a wall slide

Další implementovanou mechanikou je skákání a klouzání po stěnách, tedy wall jump a wall slide. Tyto dvě funkce spolu úzce souvisejí a výrazně rozšiřují možnosti navigace v herním světě. Wall jump umožňuje hráči opětovně vyskočit při kontaktu se stěnou ze strany, čímž se otevírá možnost postupného pohybu mezi dvěma vertikálními plochami a dosažení vyšších částí úrovně.

Základ skriptu a dědění z PlayerAbility

Skript `PlayerWall.cs` dědí z `PlayerAbility` stejně jako `PlayerDash`.

Kontrola stěny se provádí pomocí `Physics2D.OverlapCircle` s bodem `wallCheck` a poloměrem `wallCheckRadius` na příslušné vrstvě (layer). Tato metoda detekuje, zda se v okolí bodu `wallCheck` nachází collider patřící do definované vrstvy. Výsledek určuje, zda je hráč v kontaktu se stěnou.

Podobně jako u *coyote time* se používá `lastOnWallTime`, který udržuje krátký časový window po opuštění stěny. Toto umožňuje provést wall jump i krátce poté, co hráč technicky opustil stěnu, což zlepšuje herní pocit a snižuje frustrující situace.

Detekce stěny a určení jejího směru

V `Update()` se detekuje, zda se hráč dotýká stěny. Pokud ano a hráč není na zemi, nastaví se `lastOnWallTime` na `wallCoyoteTime`. Pokud ne, timer se snižuje o `Time.deltaTime`. Toto zajišťuje, že hráč má krátký čas na provedení wall jump poté, co opustil stěnu.

Metoda `DetectWallDirection()` určuje, na které straně stěna je. Existují dva přístupy: buď podle vstupu hráče (pokud hráč drží tlačítko směrem ke stěně, stěna je na této straně), nebo podle otočení postavy jako záloha. První přístup je přesnější, protože umožňuje hráči určit stěnu i když se od ní vzdaluje, druhý přístup slouží jako fallback pro situace bez inputu.

Wall slide a zpomalený pád podél stěny

V metodě `HandleWallSlide()` se kontroluje několik podmínek současně: hráč nesmí být na zemi, musí být na stěně, musí padat dolů (negativní vertikální rychlost), a musí držet pohyb směrem ke stěně. Pokud všechny podmínky platí, vertikální rychlost se ořízne na maximální hodnotu `wallSlideSpeed`. Tím vzniká efekt zpomaleného klouzání dolů.

Bez této úpravy by hráč podléhal standardní gravitaci a rychleji by padal podél stěny. `Wall slide` upravuje vertikální rychlost a poskytuje hráči časový prostor pro provedení dalšího skoku. Důležité je, že klouzání se deaktivuje během wall jump, aby nenarušovalo trajektorii odrazu.

Hodnota `wallSlideSpeed` se typicky nastavuje na nízkou hodnotu, což umožňuje pomalé klouzání bez úplného zastavení. To zajišťuje, že hráč stále padá i na stěně, ale v kontrolovaném tempu.

Wall jump a odraz od stěny

Wall jump se provádí pouze když hráč stiskne skok, nachází se na stěně a není na zemi. Skript detekuje směr stěny pomocí `DetectWallDirection()` a aplikuje sílu opačným směrem. Vertikální složka skoku je pevně nastavena na `wallJumpPowerY`, zatímco horizontální složka závisí na typu skoku.

Implementace využívá `Rigidbody2D` pro aplikaci síly metodou `AddForce()`. Vektory se nastavují podle směru stěny: pokud je stěna vpravo, horizontální složka je záporná, pokud je stěna vlevo, horizontální složka je kladná.

Tento mechanismus umožňuje hráči systematicky se odrážet mezi dvěma stěnami a dosahovat vyšších částí úrovně, kam by se normálním skokem nedostal. Wall jump je základní technikou v mnoha platformovkách a přidává hloubku do navigace herním světem.

```
void HandleWallJump()
{
    if (!pc.jumpPressedThisFrame) return;
    if (lastOnWallTime <= 0) return;
    if (pc.grounded) return;

    isWallJumping = true;
    wallJumpTimer = data.wallJumpDuration;

    int jumpDir = -wallDir;

    bool isWallToWall = true;
    RaycastHit2D hit = Physics2D.Raycast(wallCheck.position,
        Vector2.right * jumpDir, 1f, wallLayer);
    if (hit.collider != null)
        isWallToWall = false;

    float xForce = isWallToWall ?
        data.wallToWallJumpForceX :
        data.singleWallJumpForceX;
    float yForce = data.wallJumpPowerY;

    rb.linearVelocity = new Vector2(jumpDir * xForce, yForce);

    if (pc.isFacingRight != (jumpDir > 0))
        pc.CheckDirectionToFace(jumpDir > 0);

    lastOnWallTime = 0;
    isWallSliding = false;
```

```
30 }
```

Kód 4.11: Listing 11: Metoda HandleWallJump

Wall-to-wall jump a rychlý přeskok mezi stěnami

Zajímavou funkcí je rozpoznání *wall-to-wall* skoku. Pomocí `Raycast` se detekuje, zda je na druhé straně hráče další stěna v dosahu. Pokud ano, použije se vyšší horizontální síla `wallToWallJumpForceX`, což umožňuje rychlejší přeskok mezi dvěma stěnami. Pokud ne, použije se nižší `singleWallJumpForceX`.

Rozdíl mezi těmito dvěma silami je významný. Wall-to-wall jump potřebuje více horizontálního impulzu, aby hráč překonal vzdálenost mezi stěnami v krátkém čase. Single wall jump má nižší horizontální složku, jelikož cílem v tomto případě je hráče jen mírně odrazit od stěny pro vizuálně přívětivější efekt ve hře.

Tato funkce přidává do hry další úroveň kontroly a umožňuje zkušeným hráčům rychleji navigovat mezi stěnami. Rozpoznání wall-to-wall situace probíhá vždy před provedením wall jump, takže hráč nemusí manuálně přepínat mezi režimy.

Propojení s PlayerController a deaktivace vstupu

Po provedení wall jump se nastaví `isWallJumping` na true a odpočítává se timer `wallJumpTimer`. Dokud `isWallJumping` trvá, metoda `Run()` v `PlayerController` ignoruje horizontální vstup, takže hráč nemůže během wall jump reagovat na směrové tlačítko.

Po uplynutí timeru se `isWallJumping` vynuluje ve `FixedUpdate()`. Toto zajišťuje, že wall jump má předem definovanou trajektorii a hráč ji nemůže během letu měnit, což by mohlo narušit konzistenci mechaniky.

Propojení mezi `PlayerWall` a `PlayerController` je implementováno přes veřejné metody a vlastnosti. `PlayerWall` poskytuje metody jako `CanWallSlide()`, `TryWallJump()` a vlastnost `IsWallJumping`. `PlayerController` tyto metody volá v průběhu herního cyklu.

4.1.6 Střelba

Systém střelby přidává do hry další rozměr interaktivity a strategie. Na rozdíl od tradičních stříleček, kde je střelba primárním prostředkem eliminace nepřátel, v této hře slouží střelba jako záchranný mechanismus. Hráč má k dispozici pouze dva náboje, které se přebíjejí až po vystřelení obou a po krátkém časovém intervalu. Tímto designem podporujeme myšlenku, aby střelba byla pouze poslední možností záchrany a hráč se musel především soustředit na zlepšování svých dovedností v pohybu.

Systém střelby a architektura skriptů

Systém střelby se skládá ze dvou částí – skriptu na hráči `PlayerStripeShot.cs` a skriptu na projektilu `BulletStripeShot.cs`. Toto rozdělení je výhodné pro optimalizaci, protože skript projektilu se aktivuje až při výstřelu a není celou dobu aktivní v paměti.

Hráčský skript `PlayerStripeShot.cs` dědí z `PlayerAbility`, což umožňuje odemykat střelbu v průběhu hry jako speciální schopnost. Instance projektilu se provádí až při výstřelu, což šetří

výkon. Mezi výstřely skript pouze čeká na input a spravuje stav munice.

Projektily jsou vytvořeny jako prefaby v editoru a obsahují komponenty `Rigidbody2D` pro fyzikální pohyb a `Collider2D` pro detekci kolizí. Prefab také obsahuje vizuální komponenty jako sprite renderer pro grafické zobrazení projektilu.

Směr střelby a automatické zaměření na cíl

Střelba se automaticky zaměřuje na bowlingovou kouli ve scéně. To bylo zvoleno proto, aby hráč mohl využít obě ruce na klávesnici pro pohyb a další schopnosti. Výpočet směru projektilu využívá normalizovaný vektor mezi pozicí hráče a pozicí koule.

Vzorec pro výpočet směru je `(ballPosition - playerPosition).normalized()`. Tento vektor udává směr od hráče k cíli a jeho normalizace zajistí, že projektil má konstantní rychlost bez ohledu na vzdálenost.

Projektil se instancuje na pozici `firePoint`, což je bod na hráči odkud vylétá projektil. Rotace projektilu se nastaví podle směru letu pomocí `Mathf.Atan2` – tento úhel se převede na stupně a nastaví se jako rotace kolem osy Z. Toto zajišťuje, že projektil je vizuálně orientován ve směru pohybu.

```
void Shoot()
{
    Vector2 direction = (enemyTransform.position - firePoint.position).
       normalized;

    GameObject bullet = Instantiate(bulletPrefab, firePoint.position,
       Quaternion.identity);
    Rigidbody2D rb = bullet.GetComponent<Rigidbody2D>();
    rb.linearVelocity = direction * bulletSpeed;

    float rotZ = Mathf.Atan2(direction.y, direction.x) * Mathf.Rad2Deg;
    bullet.transform.rotation = Quaternion.Euler(0, 0, rotZ);
}
```

Kód 4.12: Listing 12: Metoda Shoot pro vytvoření projektilu

Vytvoření projektilu a nastavení fyzikálních parametrů

Při výstřelu se volá `Instantiate()` s prefabem projektilu. Tato metoda vytvoří novou instanci projektilu ve scéně na specified position a rotation. Následně se získá komponenta `Rigidbody2D` na nově vytvořeném objektu a nastaví se rychlost pomocí `velocity = direction * bulletSpeed`.

Projektil se nikdy nemaže manuálně po čase, vždy se zničí při kolizi s prostředím nebo koulí. Toto je implementováno v skriptu `BulletStripeShot.cs`, který detekuje kolize a volá `Destroy(gameObject)`

při kontaktu. Tento přístup šetří paměť a výkon, protože projektily, které minou cíl, nikdy nevisí ve scéně.

Omezený počet nábojů a systém přebíjení

Hráč má k dispozici pouze 2 náboje, což je záměrný designový prvek. Střelba v naší hře není primárním prostředkem eliminace nepřátel, ale poslední možností záchrany. Proměnná `maxShots` určuje kapacitu a `reloadTime` dobu přebíjení.

Po vystřelení posledního náboje se spustí korutina `Reload()`, která čeká nastavenou dobu a následně obnoví počet nábojů na maximum. Během přebíjení nelze střílet, což vytváří strategický prvek.

Tento systém podporuje myšlenku, že výstřel má být pouze poslední možností záchrany a hráč se musí soustředit na zlepšování svých dovedností v pohybu a skocích. Omezená munice nutí hráče strategicky rozhodovat, kdy použít střelbu.

Výstřel a kontrola podmínek

Metoda `TryShoot()` se volá z `PlayerController` při inputu střelby. Kontroluje se několik podmínek: zda je střelba odemčená (volání `IsUnlocked()`), zda neprobíhá přebíjení (proměnná `isReloading`) a zda jsou k dispozici náboje (`currentShots > 0`).

Pokud všechny podmínky platí, provede se výstřel voláním `Shoot()` a sníží se počet nábojů o jeden. Pokud byl vystřelen poslední náboj, spustí se přebíjení. Pokud některá z podmínek není splněna, metoda se ukončí bez akce.

Registrace inputu probíhá podobně jako u ostatních schopností. V `OnEnable()` se připojí callback na akci střelby a v `OnDisable()` se odpojí.

Respawn a obnovení stavu střelby

Při respawnu hráče se volá metoda `OnRespawn()`. Ta zastaví všechny korutiny (včetně přebíjení) a obnoví plný počet nábojů. Toto zajišťuje, že hráč má po každém respawnu čerstvou zásobu střeliva a přebíjení se nepřenáší mezi respawny.

Zastavení korutin je důležité, protože pokud by přebíjení běželo během respawnu, hráč by mohl přijít o munici, kterou by měl mít k dispozici. Vynulováním stavu se zajistí konzistentní herní podmínky po každém respawnu.

Metoda `OnRespawn()` je volána z `PlayerDeath` skriptu, který spravuje logiku smrti a respawnu hráče. Toto propojení zajišťuje, že všechny systémy hráče jsou správně resetovány při každém novém životě.

Chování projektilu po vystřelení

Skript `BulletStripeShot.cs` se nachází na instanci projektilu a aktivuje se až po vystřelení. V `OnTriggerEnter2D()` se kontroluje, s jakým objektem projektil kolidoval.

Při vstupu do trigger collideru bowlingové koule (která má tag `Ball`) se volá metoda `Stun()` na kouli. To způsobí, že koule se zpomalí a poskytne hráči čas na únik. Na rozdíl od tradičních her, kde střelba eliminuje nepřítele, v naší hře střelba pouze dočasně neutralizuje hrozbu.

Při srážce s prostředím (Collider2D na ground vrstvě) se projektil okamžitě zničí voláním `Destroy(gameObjec` Toto je implementováno pomocí layer comparison nebo tag checking, aby se rozlišilo mezi prostředím a jinými objekty.

```
public class BulletStripeShot : MonoBehaviour
{
    public bool stun = true;
    public LayerMask groundLayer;

    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("Ball"))
        {
            BallMovement ball = collision.GetComponent<BallMovement>();
            if (ball != null)
                ball.Stun(stun);

            Destroy(gameObject);
        }
    }

    private void OnCollisionEnter2D(Collision2D collision)
    {
        if (((1 << collision.gameObject.layer) & groundLayer) != 0)
            Destroy(gameObject);
    }
}
```

Kód 4.13: Listing 13: Skript BulletStripeShot

4.1.7 Bowlingová koule

Hlavním nepřátelským objektem ve hře je bowlingová koule, která představuje konstantní hrozbu pro hráče. Koule se neustále pohybuje směrem dopředu a jejím úkolem je donutit hráče neustále se pohybovat vpřed. V této sekci bude podrobně popsána implementace chování bowlingové koule, včetně systému stunu a dynamické změny rychlosti.

Přehled a základní struktura

Hlavní nepřátelský objekt ve hře, bowlingová koule, je řízen skriptem `BallMovement.cs`. Ten je přidělen na herní objekt koule a stará se o její konstantní pohyb doprava, detekci stunu a dynamický růst rychlosti. Skript vyžaduje `Rigidbody2D` a `CircleCollider2D` přes atribut `[RequireComponent]`, což zajišťuje, že jsou vždy přítomny potřebné komponenty.

Koule představuje konstantní hrozbu, která nutí hráče neustále se pohybovat vpřed. Na rozdíl od tradičních nepřátel, koule nelze zničit, pouze dočasně zpomalit. Po uplynutí stunu se koule opět zrychlí na vyšší rychlost než předtím, což vytváří progresivní obtížnost v průběhu úrovně.

Pohyb a konstantní rychlost

Koule se pohybuje konstantní rychlostí na ose X pomocí `Rigidbody2D`. V každém snímku se nastavuje `velocity.x = moveSpeed`, což zajišťuje konstantní pohyb doprava bez ohledu na kolize nebo jiné faktory. Tento přímý přístup k rychlosti je jednodušší a spolehlivější než použití sil.

Rigidbody je nastaven na `freezeRotation`, takže se koule neotáčí vlivem fyziky. Rotace animace koule se řídí samostatně v závislosti na rychlosti pohybu. Toto oddělení fyzikální rotace od vizuální animace umožňuje přesnější kontrolu nad prezentací.

```
void Awake()
{
    rb = GetComponent<Rigidbody2D>();
    rb.freezeRotation = true;
    _baseMoveSpeed = moveSpeed;
}

void FixedUpdate()
{
    if (!_canMove) return;

    Vector2 velocity = rb.linearVelocity;
    velocity.x = moveSpeed;
    rb.linearVelocity = velocity;

    UpdateAnimatorSpeed();
}

void UpdateAnimatorSpeed()
{
    if (animator != null)
    {
        float baseSpeed = 5f;
        animator.speed = Mathf.Max(0.1f, rb.linearVelocity.magnitude /
            baseSpeed);
    }
}
```

Kód 4.14: Listing 15: Pohyb bowlingové koule

Startovní prodleva a synchronizace s hráčem

Před samotným pohybem běží korutina `StartMovingAfterDelay()`, která čeká `startDelay` vteřin. Během této doby se `_canMove` nastaví na false a pohyb se neprovádí. To dává hráči čas na rozběhání před tím, než začne nebezpečí pronásledování.

Tato prodleva je důležitá pro herní design, protože bez ní by koule začala hned honit hráče, což by bylo nespravedlivé. Startovní delay poskytuje férový start a umožňuje hráči dostat se do bezpečné vzdálenosti.

Stejná prodleva se restartuje při respawnu, aby po smrti měl hráč opět čas na přípravu. To zajišťuje konzistentní herní podmínky a zabraňuje situacím, kdy by koule honila respawnujícího hráče okamžitě.

Stun systém a dočasné zpomalení

Když projektil zasáhne kouli, volá se metoda `Stun()`. Ta inkrementuje counter `_activeStuns` a spustí korutinu `StunCooldown()`. Systém rozlišuje první stun a další stuny – u prvního se uloží aktuální rychlost jako základ, u dalších se jen nastaví nízká rychlost.

Více stunů současně způsobuje, že rychlost zůstává na minimální hodnotě, dokud se všechny nestihnou vypořádat. Po uplynutí `stunDuration` se counter sníží a rychlost se obnoví na základní úroveň. Pokud je více stunů aktivních, rychlost zůstává na minimu.

Tento systém umožňuje hráči střílet více projektilů za sebou pro delší zpomalení, ale vzhledem k omezené munici (2 náboje) je toto omezeno. Navíc po každém stunu se základní rychlost zvýší, takže koule je postupně rychlejší.

```
public void Stun(bool stun)
{
    if (!stun) return;

    _activeStuns++;
    StartCoroutine(StunCooldown());
}

private IEnumerator StunCooldown()
{
    if (_activeStuns == 1)
    {
        _baseMoveSpeed = moveSpeed;
        moveSpeed = 5f;
    }
    else
    {
        moveSpeed = 5f;
    }

    yield return new WaitForSeconds(stunDuration);

    _activeStuns = Mathf.Max(0, _activeStuns - 1);
    _baseMoveSpeed = Mathf.Min(_baseMoveSpeed + speedIncreasePerStun,
       maxSpeed);

    if (_activeStuns == 0)
    {
        moveSpeed = _baseMoveSpeed;
    }
```

```
    else
    {
        moveSpeed = 5f;
    }
}
```

Kód 4.15: Listing 14: Metoda Stun a StunCooldown

Růst rychlosti a progresivní obtížnost

Důležité je, že `_baseMoveSpeed` se ukládá a dále zvyšuje s každým dalším stunem.
Proměnná `_baseMoveSpeed` se po každém stunu zvýší o `speedIncreasePerStun`, ale nikdy nepřesáhne `maxSpeed`. Pokud je více stunů aktivních současně, rychlost zůstává na minimu, dokud se všechny nestihnou vypořádat.

Tento systém vytváří progresivní obtížnost v průběhu úrovně. Každý úspěšný stun zpomalí kouli, ale po obnovení bude koule rychlejší než předtím. To nutí hráče vyhýbat se stunu, pokud není absolutně nutný, a soustředit se spíše na pohyb než na střelbu.

Maximální rychlost je omezena parametrem `maxSpeed`, aby koule nebyla příliš rychlá na to, aby ji hráč mohl uniknout. Správné nastavení tohoto parametru je klíčové pro vyvážení herní obtížnosti.

Reset při smrti hráče

Při smrti hráče se volá metoda `ResetStunAndSpeed()`. Ta zastaví všechny korutiny, vynuluje počet stunů a nastaví rychlost na předanou výchozí hodnotu. Zároveň restartuje startovací delay přes `StartMovingAfterDelay()`.

Tento reset zajišťuje, že po smrti hráče se koule vrátí do výchozího stavu. To je důležité pro férovost hry, protože hráč by neměl být penalizován za pokročilý stav koule z předchozího života. Po respawnu má hráč čas na únik a koule začíná opět od začátku.

Zastavení korutin je důležité, aby se předešlo chybám při restartu. Pokud by nějaká korutina běžela během resetu, mohla by způsobit nekonzistentní stav. Vynulování počtu stunů zajišťuje, že žádný stun nepřetrvává mezi životy.

Animace rotace a dynamická rychlost

Rychlost rotace animace koule se dynamicky mění podle aktuální rychlosti pohybu. Metoda `UpdateAnimatorSpee` počítá podíl aktuální rychlosti a základní rychlosti.
Výsledek se nastaví jako `animator.speed`, což ovlivňuje rychlost přehrávání animace.

Vzorec je `animator.speed = currentSpeed / baseSpeed`. Pokud je aktuální rychlost stejná jako základní, animator.speed je 1.0. Pokud je rychlost dvojnásobná, animator.speed je 2.0 a animace se přehrává dvakrát rychleji.

Toto zajišťuje, že vizuální rotace koule odpovídá skutečné rychlosti pohybu. Když koule stojí, rotace se zastaví. Když koule zpomalí (stun), rotace se zpomalí. Když koule zrychlí, rotace se zrychlí. Toto přispívá k celkovému vizuálnímu dojmu a konzistenci hry.

4.1.8 Další skripty nepřítele

Kromě základního pohybu bowlingové koule obsahuje nepřítel také další skripty, které rozšiřují jeho interakci s herním světem. Tyto skripty umožňují kouli ničit prostředí a reagovat na různé herní situace.

TileDestroy – přehled a účel

Skript `TileDestroy.cs` je přidělen na bowlingovou kouli a umožňuje jí ničit herní dlaždice v tilemap systému. Používá se k tomu Tilemap API, které umožňuje programově mazat dlaždice z úrovně. Tato mechanika přidává do hry dynamický prvek, kde prostředí může být poškozeno průchodem koule.

Tilemap systém v Unity ukládá dlaždice do gridu a umožňuje efektivní správu velkých herních světů. `TileDestroy` využívá `GetComponent<Tilemap>()` pro přístup k tilemap a `SetTile()` pro nastavení dlaždice na null (smazání).

Tato funkcionalita je klíčová pro herní design, protože vytváří časový tlak na hráče. Pokud koule dlouho stojí za hráčem, může zničit platformy a zablokovat cestu. Hráč musí neustále postupovat vpřed, aby si udržel cestu otevřenou.

Tick-based ničení dlaždic a optimalizace

Ničení dlaždic probíhá v `FixedUpdate()` v taktovacích tikách, nikoli v každém snímku. Proměnná `timer` odpočítává čas mezi ticky, který je definován parametrem `destroyCooldown`. Pokud timer klesne k nule, začne se iterovat přes všechny kolize a dlaždice, které koule prochází, se mažou.

Důvod pro tick-based přístup je optimalizace. Kontrola dlaždic a jejich mazání je výpočetně náročná operace, zejména při vysoké rychlosti koule. Provedením této operace pouze jednou za několik snímků se výrazně sníží procesorová zátěž, aniž by to mělo viditelný dopad na herní zážitek.

Po dokončení ticku se timer resetuje na hodnotu `destroyCooldown` a cyklus pokračuje. Toto zajišťuje konzistentní chování bez ohledu na rychlost koule nebo snímkovou frekvenci.

Limit dlaždic na tick a výkonová ochrana

Aby se předešlo propadům výkonu při vysoké rychlosti koule, existuje parametr `maxTilesPerTick`. V každém tickovi se zničí maximálně tento počet dlaždic. Jakmile je limit dosažen, další dlaždice se mažou až v dalším tickovi. Parametr `destroyCooldown` určuje, jak často tick probíhá.

Toto omezení je důležité při vysoké rychlosti koule, kdy může projít stovkami dlaždic za sekundu. Bez limitu by to mohlo způsobit výrazné zpomalení hry. S limitem je zajištěno, že i při vysoké rychlosti zůstává výkon konzistentní.

Hodnota `maxTilesPerTick` se typicky nastavuje na 50-100 v závislosti na výkonu targetované platformy a typické rychlosti koule. Nižší hodnota = lepší výkon, vyšší hodnota = rychlejší ničení prostředí.

Bounds a WorldToCell – převod souřadnic

Pro každou kolizi se počítají `Bounds` a převádějí na souřadnice tilemap buněk pomocí `WorldToCell()`. `Bounds` representují ohraničující box collideru, zatímko `WorldToCell()` převádí world space souřadnice na grid space souřadnice tilemap.

Iteruje se přes všechny buňky v rozsahu bounds pomocí vnořených smyček. Pro každou buňku se zkontroluje, zda dlaždice existuje voláním `tilemap.HasTile(cell)`. Pokud dlaždice existuje, nastaví se na `null` voláním `tilemap.SetTile(cell, null)`.

Tento přístup zajišťuje, že všechny dlaždice v kontaktu s koulí jsou zničeny. Převod mezi world space a cell space je základní operací Tilemap API a umožňuje přesnou kontrolu nad ničením dlaždic.

DestroyOnTouch – přehled a funkčnost

Skript `DestroyOnTouch.cs` řeší okamžité zničení objektů s tagem `Destroyable` při kontaktu s koulí. Na rozdíl od `TileDestroy`, který neničí dlaždice, ale celé herní objekty. Tento skript umožňuje kouli ničit různé objekty ve scéně jako jsou bedny, rozbitné předměty nebo dekorace.

Implementace využívá `OnTriggerEnter2D()` pro detekci vstupu do trigger collideru. Když koule vstoupí do triggeru jiného objektu, kontroluje se tag tohoto objektu. Pokud tag odpovídá `Destroyable`, objekt se zničí.

Přístup s triggery je vhodný pro objekty, které nepotřebují fyzikální kolizi. Trigger collidery detekují přítomnost bez fyzické interakce, což je užitečné pro objekty, které mají být zničeny, ale nemají ovlivňovat pohyb koule.

Závěr

V této práci jsem navrhl a implementoval herní mechaniky pro 2D plošinovku v Unity. Klíčovým prvkem bylo vytvoření komplexního systému pohybu hráče zahrnujícího pohyb, pokročilé skokové mechaniky (coyote time, jump buffer, wall jump, variable jump height) a mechaniku dashe.

Důležitým aspektem bylo také vytvoření vlastního fyzikálního systému namísto spoléhání se na Unity vestavěné nástroje. Toto řešení umožňuje dynamickou úpravu gravitace podle aktuálního stavu hráče, což zásadně zlepšuje herní pocit.

Systém střelby byl navržen jako záchranný mechanismus s omezenou municí, což podporuje herní filosofii zaměřenou na zlepšování pohybových dovedností.

Bowlingová koule jako nepřítel poskytuje konstantní hrozbu s progresivní obtížností díky stun systému a růstu rychlosti.

Seznam obrázků